SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1
Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management.

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	**G.Yashmita**	Register Number	**192511032**
Department / Section	CSE	Academic Year / Semester	2026
Faculty Name	Dr.Arul Raj A.M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	04-11-2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Existing Data Centre and Requirements	3		Definition; characteristics; deployment models; scenario mapping
2	Virtualization Fundamentals and Hypervisors	3		Timeline; hardware evolution; relevance to university
3	Server Consolidation and Resource Calculations	3		Web/software evolution; SOA; APIs; scenario relevance
4	Virtual Machines and Containers	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Virtualized Storage and Network Infrastructure	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Resource Allocation, Scheduling and Monitoring	5		Responsibility layers; five-application mapping; comparison; providers
7	Availability, Security and Disaster Recovery	3		Four or more variants; two justified choices; provider risks
8	Virtualized Cloud Data Centre Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Validation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Cost, Performance and Final Recommendation	7		Three-way comparison; 3-year costs; risks; phased roadmap

11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations		Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution		Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices		Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification		Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots		Local API execution, actual functional-test output and local process metrics are included. Docker container execution and docker stats remain unverified.
10	Cost assumptions, 3-year totals, risk controls and migration roadmap		Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation		Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code		Python standard-library REST API, Dockerfile and Docker Compose configuration are provided.
Virtualization		Docker CPU/memory limits are configured; a running container was not verified because Docker Engine was unavailable.
PaaS or SaaS		Managed web-service deployment procedure described; live deployment not verified
Functional test		Actual local functional tests passed: HTTP 200 responses and HTTP 404 for a missing student.
Failure case		Authentic local API, test, configuration and local process-metric screenshots are inserted below; Docker runtime screenshots are not available.

Screenshots		Screenshot requirements identified; authentic screenshots need verification
Reproducibility		Build commands, run commands, endpoints and expected results specified

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping		All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance		Well explained with relevant examples
3	Internet software evolution, SOA and APIs		Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations		Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs		API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping		All five applications categorized with justification
7	XaaS variants and two justified selections		Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery		Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots		Local API execution and actual test screenshots are included; running Docker-container and docker stats evidence is not available.
10	Three-year cost comparison, risks and migration roadmap		Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation		Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code		Python standard-library REST API and Dockerfile provided.
Virtualization		Docker build/run commands and Compose resource limits provided; container execution not verified.
PaaS or SaaS		Cloud deployment steps described; actual deployment confirmation required
Functional test		Actual local output: health/list/existing student HTTP 200; missing student HTTP 404.

Failure case		Actual local screenshots inserted; Docker runtime and docker stats evidence not captured.
Screenshots		Required screenshots identified but authenticity and completeness not verified
Reproducibility		The application code, Docker configuration and test procedure are consistent; the API uses Python standard-library HTTP server, not Flask.
Interpretation		Technical outcomes and implementation limitations explained
Code and report consistency		Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☐ Yes ☐ Partly ☐ No
☐ Existing data-centre requirements and limitations are identified.	☐Yes ☐ Partly ☐ No
☐ Virtualization and hypervisor types are explained.	☐Yes ☐ Partly ☐ No
☐ Server consolidation calculations include working and assumptions.	☐Yes ☐ Partly ☐ No
☐ VM and container comparison and workload mapping are included.	☐Yes ☐ Partly ☐ No
☐ Storage and network virtualization are addressed.	☐ Yes ☐Partly ☐ No
☐ Resource allocation and monitoring policies are included.	☐Yes ☐ Partly ☐ No
☐ Security risks, backup and disaster recovery are discussed.	☐Yes ☐ Partly ☐ No
☐ A labelled virtualized data-centre architecture diagram is included.	☐ Yes ☐Partly ☐ No
☐ Implementation screenshots and actual test evidence are provided.	☐ Yes ☐ Partly ☐ No
☐ Three-year cost analysis and migration roadmap are included.	☐ Yes ☐ Partly ☐ No

GENERAL INSTRUCTIONS

- Use clear headings and answer each task in the order given.
- Include diagrams, tables, calculations and screenshots wherever requested.
- State assumptions and units for all numerical calculations.
- Use credible technical references and cite sources consistently.
- Ensure that implementation evidence reflects work actually performed by the team.
- Submit the report as a well-formatted document with a title page, contents, references and appendices where required.

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

CSA1520 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title:

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management

(Total Marks – 50) Duration: 1 week

Scenario – University Data Centre Virtualization

A growing university operates an on-premises data centre that hosts its Student Information System, Learning Management System, Library Management System, Examination Portal, research applications and institutional website. Each major application runs on a separate physical server. Server resources are underutilized during normal periods, while admission, examination and result-publication periods create high demand. The university experiences rising electricity and maintenance costs, slow hardware procurement, limited scalability and inadequate disaster-recovery arrangements.

The current data-centre environment is assumed to have the following characteristics:

- 20 physical servers, each with 16 CPU cores, 64 GB RAM and 2 TB local storage.
- Average CPU utilisation is approximately 20%; peak utilisation reaches 90–95% during admission and examination periods.
- Most servers are dedicated to individual applications and cannot share unused capacity efficiently.
- There is no centralized virtual-machine (VM) management platform and no tested live-migration process.
- Backup is performed periodically, but a separate disaster-recovery site and documented recovery objectives are not available.
- The university wants to reduce infrastructure costs while improving availability, scalability, security and resource utilization.

You are appointed as a cloud infrastructure consultant. Analyse the existing environment and design a virtualized cloud data-centre solution. Your proposal must explain the role of hypervisors, virtual machines, containers, storage and networking virtualization, resource allocation, monitoring, security and disaster recovery. Justify all major decisions using calculations, diagrams and practical evidence.

Design Challenge:

On Day 4, the university announces that it will introduce a new digital learning platform and extend selected services to 10 affiliated colleges. The expected peak workload will increase to five times the current peak. The university requires at least 99.9% service availability, stronger tenant isolation and controlled operating costs. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Analyse the Existing Data Centre and Identify Requirements (3 Marks)

Study the current data-centre environment and identify the reasons for adopting virtualization.

Explain the purpose of a data centre and identify its major components: compute, storage, networking, power, cooling and management.
Identify at least four limitations of the existing physical-server environment.
Define resource utilization, server consolidation, scalability, availability and fault tolerance.
Prepare a requirement table covering performance, capacity, security, availability and cost.
Evidence Expected: A concise assessment of the existing environment, clearly linking each identified problem to a virtualization or cloud-infrastructure requirement.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors (4 Marks)

Explain the technologies that enable multiple computing environments to share physical infrastructure.
Define virtualization and explain its purpose in a cloud data centre.
Explain the role and working of a hypervisor.
Differentiate Type-1 (bare-metal) and Type-2 (hosted) hypervisors with suitable examples such as VMware ESXi, Microsoft Hyper-V, KVM and Oracle VirtualBox.
Compare full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization (containers).
Recommend a suitable hypervisor for the university and justify the selection.
Evidence Expected: A labelled virtualization architecture diagram, comparison table and justified hypervisor recommendation.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements (5 Marks)

Propose a consolidation plan to reduce the number of physical servers while maintaining sufficient capacity.
Explain physical-to-virtual (P2V) migration and server consolidation.
Estimate the total CPU and memory demand using the stated server specifications and average utilization. State all assumptions.
Assume a target average CPU utilization of 65% on the consolidated hosts.
Calculate the minimum number of hosts required based on CPU capacity, then check whether memory capacity changes the result.
Include a reasonable capacity reserve for peak demand and host failure; show the calculation clearly.
Discuss resource overcommitment, CPU scheduling, memory ballooning and the noisy-neighbour problem.
Evidence Expected: Worked calculations showing CPU and memory requirements, assumptions, capacity reserve and the recommended number of physical hosts. A final number without working will not receive full credit.

TASK 4 – Compare Virtual Machines and Containers (4 Marks)

Determine which workloads should run in virtual machines and which can run in containers.
Explain the architecture and characteristics of virtual machines and containers.
Compare VM and container isolation, operating-system requirements, startup time, resource overhead, portability and management.

Map at least five university workloads (for example, database, legacy student system, web portal, learning platform and analytics service) to VMs or containers.
Explain how Docker and a container orchestration platform such as Kubernetes can support deployment and scaling.
Identify situations where a VM is more appropriate than a container.
Evidence Expected: A VM-versus-container comparison and a workload mapping table supported by technical justification.

TASK 5 – Design Virtualized Storage and Network Infrastructure (4 Marks)

Design storage and network resources that support the proposed virtualized data centre.
Explain storage virtualization and compare direct-attached storage (DAS), network-attached storage (NAS) and storage area networks (SAN).
Explain software-defined storage (SDS), thin provisioning, snapshots, replication and backup.
Explain network virtualization, virtual switches, VLANs, virtual networks and software-defined networking (SDN).
Propose separate logical network segments for administration, student services, databases and management traffic.
Recommend suitable storage and network approaches for performance, security and availability.
Evidence Expected: A labelled storage and network design with explanations of the selected technologies and their roles.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring (5 Marks)

Develop a resource-management strategy that allocates infrastructure capacity efficiently.
Explain CPU, memory, storage and network resource allocation to VMs and containers.
Compare static allocation and dynamic allocation, including resource reservations, limits and shares.
Explain load balancing, workload scheduling, auto-scaling and high availability.
Identify suitable performance indicators: CPU utilization, memory utilization, disk I/O, network throughput, latency and VM availability.
Propose monitoring and alerting methods, and explain how the administrator should respond to resource contention.
Evidence Expected: A resource-allocation policy table and a monitoring plan that includes measurable indicators, thresholds and corrective actions.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery (4 Marks)

Identify the risks associated with a virtualized data centre and propose suitable controls.
Explain hypervisor security, VM isolation, secure VM images, patch management and privileged access control.
Discuss VM escape, VM-to-VM attacks, insecure snapshots, VM sprawl and denial-of-service risks.

Recommend network segmentation, encryption, multi-factor authentication, role-based access control and audit logging.
Explain backup, replication, snapshots, live migration and disaster recovery.
Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO), and propose suitable targets for at least two university applications.
Evidence Expected: A risk-and-mitigation table and a disaster-recovery plan with justified RTO and RPO values.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture (5 Marks)

Construct an end-to-end architecture integrating compute, storage, network, virtualization and management components.
Draw a labelled architecture showing users, internet or campus network, firewall, load balancer, physical hosts, hypervisor layer, VMs/containers, virtual network, storage and monitoring.
Show how the university applications are distributed across hosts and how shared resources are allocated.
Include a management plane for provisioning, monitoring, logging and policy enforcement.
Show backup or replication to a secondary site or cloud region.
Justify the architecture in terms of scalability, performance, fault tolerance, security and cost.
Evidence Expected: A complete, clearly labelled architecture diagram and a written justification for each major component and design decision.

TASK 9 – Hands-on Implementation and Validation (5 Marks)

Build a small proof-of-concept that demonstrates virtualization and resource management.
Create at least one VM using VirtualBox, VMware, Hyper-V or KVM, or create a container using Docker.
Deploy a simple university-related service, such as a web page, student-record API or database-backed application.
Configure CPU, memory and storage resources and record the selected values.
Perform functional tests and at least one resource or load test. Record the observations.
Capture screenshots of the VM/container configuration, running service, resource-monitoring output and test results.
Document installation steps, commands, configuration details, issues faced and how they were resolved.
Evidence Expected: Reproducible implementation steps, source/configuration files, annotated screenshots and actual test evidence. Do not present simulated results as actual execution.

TASK 10 – Evaluate Cost, Performance and Final Recommendation (3 Marks)

Evaluate the proposed solution against the existing physical data-centre arrangement.

Compare the current physical-server model and proposed virtualized model in terms of hardware cost, electricity, cooling, maintenance, licensing and administration effort.
Prepare a simple three-year cost comparison using clearly stated assumptions.
Separate one-time capital expenditure from recurring operational expenditure.
Evaluate expected improvements in utilization, scalability, availability and manageability.
Identify risks such as licensing costs, vendor lock-in, skills requirements, hardware failure and migration downtime.
Provide a phased migration roadmap, identifying which applications should be migrated first and why.
Evidence Expected: A three-year cost table, risk analysis, migration roadmap and evidence-based final recommendation.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure (8 Marks)

Changed Requirement

The university introduces a digital learning platform and extends selected services to 10 affiliated colleges. The expected peak workload increases to five times the current peak. The system must achieve 99.9% availability, provide secure isolation between colleges and remain within a limited operating budget.
Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN the proposed infrastructure.

Impact Analysis – 2 Marks

Identify changes in workload, users, storage, network traffic and availability requirements.
Explain how multi-tenant access affects VM/container isolation, network segmentation and access control.
Identify which original capacity and cost assumptions are no longer valid.

Capacity and Availability Evaluation – 2 Marks

Estimate the additional CPU, memory, storage and network capacity needed for the five-times peak workload.
Evaluate whether the proposed number of hosts and the existing failover arrangement are sufficient.
Explain the meaning of 99.9% availability (approximately 8.76 hours of downtime per year) and identify measures needed to approach it.

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the original infrastructure and justify the decision.
Describe the revised architecture. Consider clustering, live migration, auto-scaling, container orchestration, multi-zone deployment or hybrid-cloud capacity where appropriate.

Explain changes to resource-allocation policies, tenant isolation, backup and disaster recovery.

Evidence-Based Defence – 2 Marks

Support the revised design with capacity calculations and clearly stated assumptions.
Compare the cost and operational complexity of the original and revised designs.
Discuss trade-offs involving performance, availability, security, licensing and management effort.
Evidence Expected: A technically justified decision supported by revised architecture, capacity calculations, availability considerations, security controls and cost comparison.
Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks
Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks
Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks
Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks
Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Existing Data Centre and Requirements	L2 – Understand / Analyse	3
Task 2	Virtualization Fundamentals and Hypervisors	L2 – Understand	3
Task 3	Server Consolidation and Resource Calculations	L3 – Apply	3
Task 4	Virtual Machines and Containers	L4 – Analyse	5
Task 5	Virtualized Storage and Network Infrastructure	L3/L4 – Apply / Analyse	4
Task 6	Resource Allocation, Scheduling and Monitoring	L4 – Analyse	5
Task 7	Availability, Security and Disaster Recovery	L4 – Analyse	3
Task 8	Virtualized Cloud Data Centre Architecture	L6 – Create	5
Task 9	Hands-on Implementation and Validation	L3 – Apply	4
Task 10	Cost, Performance and Final Recommendation	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4/L5/L6 – Analyse / Evaluate / Create	8
	TOTAL		50

ANSWER

GITHUB LINK: https://github.com/192511032-YashmitaG/CSA1520-Cloud-Computing-A.git

TASK 1 – Analyse the Existing Data Centre and Identify Requirements

1.1 Purpose of a Data Centre

A data centre is a centralized facility that contains computing, storage, networking and supporting infrastructure required to store, process, manage and distribute data and applications. It provides the hardware and software environment needed to operate an organization's digital services securely and reliably.

In the university, the data centre supports the following applications:

- Student Information System (SIS)
- Learning Management System (LMS)
- Library Management System (LMS)
- Examination Portal
- Research Applications
- Institutional Website

These services require continuous access to computing resources, secure data storage, network connectivity and reliable power.

1.2 Major Components of a Data Centre

Component	Purpose	Application in the University
Compute	Processes application instructions using CPUs and memory.	Runs student, examination and learning applications.
Storage	Stores databases, documents, backups and application files.	Stores student records, examination results and library data.
Networking	Connects servers, users, storage and external networks.	Provides access to students, faculty and affiliated colleges.
Power	Supplies continuous electrical power to equipment.	Uses UPS systems and backup generators to prevent service interruption.
Cooling	Maintains suitable operating temperatures.	Prevents servers from overheating.
Management	Monitors and controls infrastructure resources.	Supports server administration, security, backup and performance monitoring.

1.3 Limitations of the Existing Physical-Server Environment

The university currently operates 20 physical servers. Each server has 16 CPU cores, 64 GB RAM and 2 TB local storage.

The existing infrastructure has the following limitations:

1. Low resource utilization: Average CPU utilization is approximately 20%. A significant amount of processing capacity remains unused during normal periods.

2. Limited scalability: Each application depends on a dedicated physical server. Increasing capacity requires purchasing and installing additional hardware.
3. High operational cost: Maintaining multiple physical servers increases electricity, cooling, maintenance and hardware replacement expenses.
4. Lack of centralized management: There is no centralized virtual-machine management platform, making resource allocation and administration more difficult.
5. Limited availability: If a physical server fails, the application hosted on that server may become unavailable.
6. Inadequate disaster recovery: Periodic backups are performed, but a separate disaster-recovery site and documented recovery objectives are not available.
7. Poor workload flexibility: Unused resources on one physical server cannot easily be shared with another application.

1.4 Reasons for Adopting Virtualization

Virtualization allows multiple virtual machines to operate on a single physical server. Each VM can run its own operating system and applications while sharing the underlying hardware.

The university should adopt virtualization for the following reasons:

- Server consolidation: Multiple application servers can be hosted on fewer physical machines.
- Improved utilization: CPU, memory and storage resources can be allocated according to application requirements.
- Scalability: Virtual machines can be created, resized or migrated more easily than purchasing new physical servers.
- Availability: Virtual machines can be restarted or migrated to another host when hardware problems occur, subject to the configured infrastructure.
- Centralized management: Administrators can monitor and manage virtual machines from a central platform.
- Cost reduction: Consolidation can reduce hardware, electricity, cooling and maintenance requirements.

1.5 Important Definitions

Term	Definition
Resource Utilization	The percentage of available CPU, memory, storage or network capacity currently being used.
Server Consolidation	Combining workloads from multiple physical servers onto fewer physical servers using virtualization.
Scalability	The ability of an infrastructure to increase or decrease resources according to workload requirements.
Availability	The percentage of time a system remains operational and accessible.
Fault Tolerance	The ability of a system to continue operating when one or more components fail.

1.6 Requirement Analysis

Requirement	Existing Problem	Proposed Solution
Performance	High CPU demand during admission and examination periods.	Resource allocation, load balancing and workload monitoring.
Capacity	Dedicated servers cannot efficiently share unused resources.	Server consolidation using hypervisors and VMs.
Security	Applications and users require controlled access to resources.	Network segmentation, encryption, MFA and RBAC.

Availability	Hardware failure can interrupt services.	Host clustering, failover and redundant infrastructure.
Cost	High electricity, cooling and maintenance expenses.	Virtualization and efficient resource utilization.

Conclusion: The existing physical-server environment has low average resource utilization and limited scalability. A virtualized data centre can improve resource utilization, centralized management, availability and operational efficiency.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors

2.1 Definition of Virtualization

Virtualization is a technology that creates virtual versions of computing resources such as servers, operating systems, storage and networks. It allows multiple isolated computing environments to share the resources of a single physical machine.

In a cloud data centre, virtualization enables administrators to create and manage multiple virtual machines on shared physical infrastructure.

2.2 Purpose of Virtualization in a Cloud Data Centre

The major purposes of virtualization are:

1. To reduce the number of physical servers.
2. To improve hardware resource utilization.
3. To provide isolated environments for different applications.
4. To simplify infrastructure management.
5. To support workload migration and scalability.
6. To reduce infrastructure operating costs.

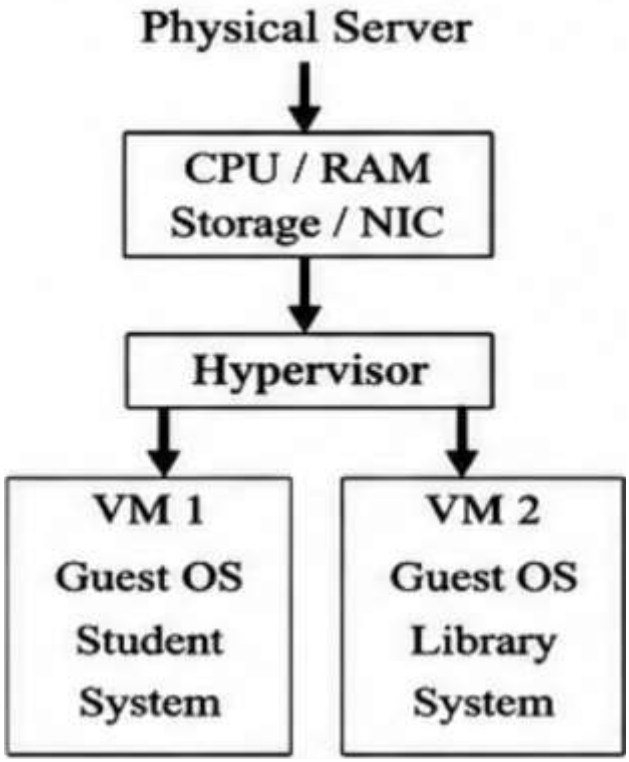
2.3 Role and Working of a Hypervisor

A hypervisor is software or a virtualization layer that creates, runs and manages virtual machines. It distributes physical hardware resources such as CPU, RAM, storage and network interfaces among virtual machines.

Working process:

1. The physical server provides CPU, memory, storage and network resources.
2. The hypervisor runs directly on the hardware or on top of a host operating system.
3. The administrator creates virtual machines and assigns resources to them.
4. Each VM receives virtual CPU, memory, disk and network interfaces.
5. The guest operating system runs inside each VM.
6. Applications execute inside the guest operating system.

2.4 Labelled Virtualization Architecture



2.5 Types of Hypervisors

There are two major types of hypervisors.

Type-1 Hypervisor – Bare-Metal

A Type-1 hypervisor runs directly on the physical server hardware without requiring a general-purpose host operating system underneath it.

Examples:

- VMware ESXi
- Microsoft Hyper-V
- KVM (Kernel-based Virtual Machine)

Advantages:

- High performance.
- Better control over hardware resources.
- Suitable for enterprise data centres.
- Supports centralized VM management and high-availability configurations when combined with appropriate management tools.

Type-2 Hypervisor – Hosted

A Type-2 hypervisor runs as an application on top of an existing operating system.

Examples:

- Oracle VirtualBox

- VMware Workstation

Advantages:

- Easy to install and configure.
- Suitable for learning, development and testing.
- Does not require dedicated server **hardware.**

2.6 Type-1 and Type-2 Comparison

Feature	Type-1 Hypervisor	Type-2 Hypervisor
Installation	Directly on hardware	On a host operating system
Performance	Generally higher	Has additional host OS overhead
Security	Smaller dependency on a general-purpose host OS	Depends partly on host OS security
Usage	Enterprise data centres	Desktop testing and development
Examples	ESXi, Hyper-V, KVM	VirtualBox, VMware Workstation

2.7 Comparison of Virtualization Techniques

Technique	Explanation	Example
Full Virtualization	Provides a complete virtual hardware environment so an unmodified guest OS can run inside a VM.	VMware ESXi
Para-Virtualization	Uses virtualization-aware guest components or interfaces to improve communication with the hypervisor.	Xen PV
Hardware-Assisted Virtualization	Uses CPU features such as Intel VT-x and AMD-V to support virtualization.	Modern KVM and ESXi deployments
OS-Level Virtualization	Runs isolated containers while sharing the host operating-system kernel.	Docker

2.8 Recommended Hypervisor for the University

The recommended solution is KVM or VMware ESXi, depending on the university's budget, licensing requirements and available technical expertise.

For a cost-conscious university, KVM is a suitable choice because it is integrated into the Linux kernel and can be managed using tools such as libvirt and Proxmox VE.

For an organization that prefers a commercial virtualization ecosystem and has the required licensing budget, VMware ESXi may also be considered.

Final recommendation: KVM-based virtualization is recommended for the proposed cost-conscious design. It provides enterprise virtualization capabilities without requiring the same commercial hypervisor licensing model as proprietary alternatives. The university must still budget for hardware, support, backup and management.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements

3.1 Meaning of Server Consolidation

Server consolidation is the process of combining workloads from multiple physical servers onto fewer, more powerful physical servers using virtualization.

Instead of running one application on each physical server, multiple virtual machines can run on a single physical host.

This improves hardware utilization and reduces infrastructure complexity.

3.2 Physical-to-Virtual (P2V) Migration

P2V migration is the process of converting an existing physical server into a virtual machine.

Migration steps:

1. Assess the physical server's CPU, RAM, storage and application dependencies.
2. Take a verified backup of the server.
3. Use a P2V conversion tool to capture the operating system, applications and data.
4. Create a virtual machine on the selected hypervisor.
5. Assign suitable virtual CPU, memory and storage resources.
6. Transfer the converted system to the virtual environment.
7. Test application functionality and performance.
8. Switch production traffic after successful validation.

3.3 Existing Infrastructure Details

Given data:

- Number of physical servers = 20
- CPU cores per server = 16
- RAM per server = 64 GB
- Storage per server = 2 TB
- Average CPU utilization = 20%
- Peak CPU utilization = 90–95%
- Target average CPU utilization on consolidated hosts = 65%

3.4 Total Existing CPU Capacity

$$\text{Total CPU Cores} = 20 \times 16$$
$$= 320 \text{ cores}$$

Average CPU demand:

$$320 \times \frac{20}{100} = 64 \text{ cores}$$

Therefore, the average CPU demand is 64 core-equivalents.

3.5 Total Existing Memory Capacity

$$\text{Total RAM} = 20 \times 64$$
$$= 1280 \text{ GB}$$

The assignment does not provide average memory utilization. Therefore, an assumption is required to estimate memory demand.

Assumption: Average memory utilization is 60%.

$$\text{Average Memory Demand} = 1280 \times 0.60$$
$$= 768 \text{ GB}$$

The estimated average memory demand is 768 GB. This is an assumption for planning, not a value supplied by the assignment.

3.6 Proposed Consolidated Host Configuration

For calculation purposes, assume each new physical host has:

- 32 CPU cores
- 256 GB RAM
- 10 TB usable local or allocated storage capacity

The university should use shared storage for workloads that require VM migration and host-level failover.

3.7 CPU-Based Host Calculation

Target CPU utilization = 65%.

Usable CPU capacity per host:

$$32 \times 0.65 = 20.8 \text{ cores}$$

Number of hosts required for average CPU demand:

$$N = \frac{64}{20.8}$$

$$N = 3.08$$

Rounding up:

$$\boxed{N = 4 \text{ hosts}}$$

Therefore, four hosts are required to support the estimated average CPU demand at the target utilization.

3.8 Memory-Based Host Calculation

Assumed average memory demand = 768 GB.

To maintain a 25% memory reserve, usable memory per host is:

$$256 \times 0.75 = 192 \text{ GB}$$

Number of hosts required:

$$N = \frac{768}{192}$$

$$\boxed{N = 4 \text{ hosts}}$$

Therefore, memory demand also requires four hosts under the stated assumptions.

3.9 Peak-Demand and Host-Failure Reserve

The assignment states that peak CPU utilization reaches 90–95%.

At 95% peak utilization:

$$320 \times 0.95 = 304 \text{ cores}$$

Number of active hosts required at 65% target utilization:

$$N = \frac{304}{20.8} = 14.62$$

Rounding up:

$$N = 15 \text{ active hosts}$$

To support one host failure while maintaining the required capacity, add one additional host:

$$15 + 1 = \boxed{16 \text{ physical hosts}}$$

After one host fails, 15 hosts remain.

Their usable CPU capacity is:

$$15 \times 32 \times 0.65 = 312 \text{ cores}$$

Since 312 cores are greater than the estimated peak demand of 304 cores, the design can support the stated peak CPU demand after one host failure, subject to the assumptions.

3.10 Consolidation Calculation Summary

Parameter	Result
Existing physical servers	20
Existing total CPU capacity	320 cores
Average CPU demand	64 cores
Assumed average memory demand	768 GB
CPU-based hosts for average demand	4
Memory-based hosts for average demand	4
Active hosts for 95% peak demand	15
Additional N+1 reserve	1 host
Recommended peak-capacity design	16 hosts

The average-demand calculation suggests that four hosts can support normal workloads. However, this number is not sufficient for the stated 90–95% peak demand. Therefore, the recommended design uses 16 hosts if the university intends to accommodate the full existing peak without relying on cloud bursting or reducing workloads.

3.11 Resource Overcommitment

Resource overcommitment occurs when the total virtual resources assigned to VMs exceed the physical resources available, based on the expectation that all workloads will not use their maximum allocation simultaneously.

For example, a server with 32 physical CPU cores may be assigned 48 virtual CPUs across multiple VMs.

Overcommitment can improve utilization, but excessive overcommitment may cause performance degradation during peak periods.

3.12 CPU Scheduling

CPU scheduling is the process of distributing available physical CPU processing time among virtual machines.

The hypervisor schedules virtual CPUs onto physical CPU cores according to configured reservations, limits and shares.

3.13 Memory Ballooning

Memory ballooning is a memory-management technique in which a hypervisor reclaims unused memory from virtual machines when the host experiences memory pressure.

A balloon driver inside the guest operating system helps the hypervisor reclaim memory safely where possible.

3.14 Noisy-Neighbour Problem

The noisy-neighbour problem occurs when one VM or container consumes excessive shared resources, affecting the performance of other workloads hosted on the same infrastructure.

Control measures:

- Configure CPU and memory limits.
- Use resource reservations for critical applications.
- Monitor host-level resource consumption.
- Separate high-demand workloads.
- Apply workload scheduling and load balancing.

Conclusion: Four hosts are sufficient for the assumed average demand, but 16 hosts are required to support the existing 95% peak CPU demand while retaining one-host failure capacity under the stated planning assumptions.

TASK 4 – Compare Virtual Machines and Containers

4.1 Virtual Machines

A virtual machine is a software-based computer that runs an independent guest operating system on virtualized hardware.

Each VM has virtual CPU, memory, storage and network interfaces.

Characteristics:

- Runs its own guest operating system.
- Provides strong isolation between workloads.
- Can run different operating systems on the same physical host.
- Requires more resources than a typical container.
- Suitable for legacy and operating-system-dependent applications.

4.2 Containers

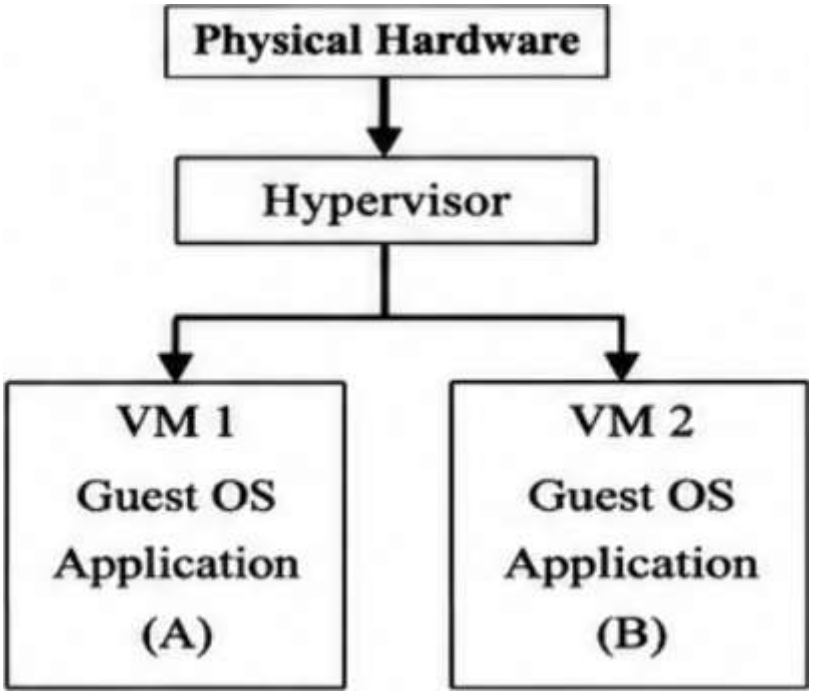
A container is a lightweight isolated environment used to run an application and its dependencies. Containers share the host operating-system kernel.

Docker is a widely used platform for building and running containers.

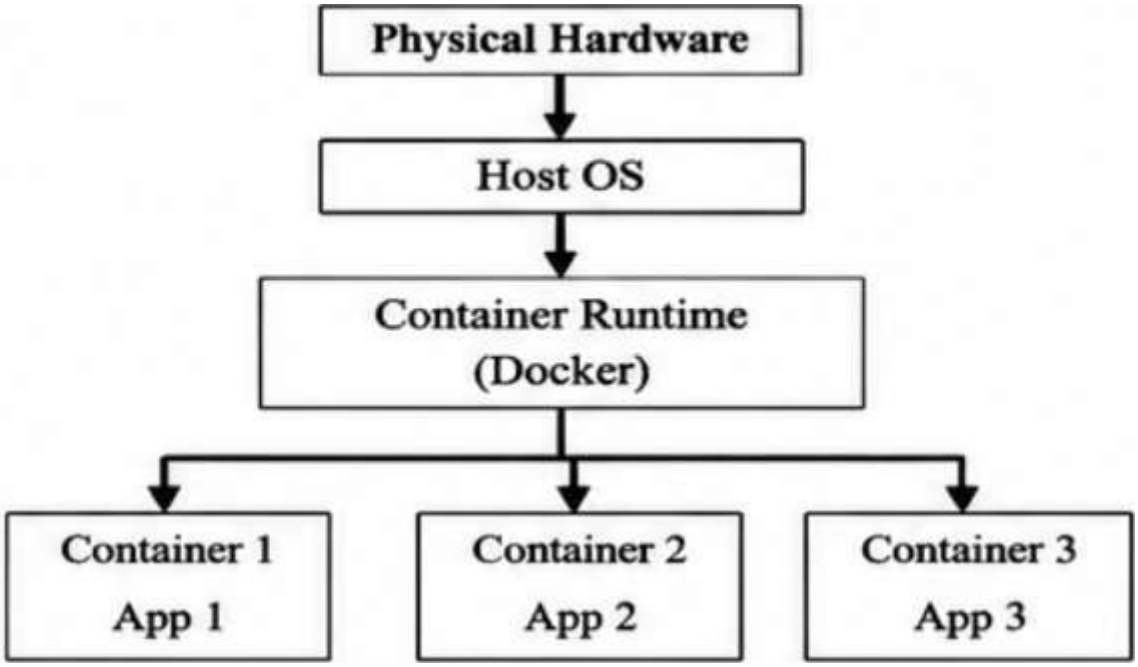
Characteristics:

- Shares the host OS kernel.
- Starts quickly.
- Uses fewer resources than a full VM in many workloads.
- Supports application portability.
- Suitable for microservices and web applications.

4.3 VM Architecture



4.4 Container Architecture



4.5 VM and Container Comparison

Feature	Virtual Machine	Container
Operating System	Each VM has its own guest OS.	Shares the host OS kernel.
Isolation	Hardware virtualization provides strong isolation.	Process and namespace isolation; kernel is shared.
Startup Time	Usually slower.	Usually faster.
Resource Overhead	Higher because each VM includes a guest OS.	Generally lower.
Portability	VM images can be moved between compatible hypervisors.	Container images can be deployed on compatible container runtimes.
Management	Managed using hypervisor platforms.	Managed using container runtimes and orchestration platforms.
Suitable Workloads	Legacy systems, databases and OS-specific applications.	Microservices, web APIs and scalable application services.

4.6 University Workload Mapping

University Workload	Recommended Platform	Justification
Student Information System	VM	May depend on legacy operating-system and application configurations.
Database Server	VM	Provides dedicated resource allocation and isolation for important database workloads.
Institutional Website	Container	Web application components can be deployed and scaled independently.
Learning Management System	Containers	Supports horizontal scaling of web and application services.
Examination Portal	Containers	Allows multiple application instances to handle examination-period traffic.
Research Applications	VM or Container	Selection depends on operating-system, hardware and dependency requirements.

4.7 Role of Docker

Docker is a containerization platform used to build, package and run applications in containers.

It helps the university by:

- Packaging applications with their dependencies.
- Maintaining consistent development and production environments.
- Simplifying application deployment.
- Supporting repeatable application releases.
- Reducing deployment differences between environments.

4.8 Role of Kubernetes

Kubernetes is a container orchestration platform that automates the deployment, scaling and management of containerized applications.

Its important features include:

- Automatic container scheduling.
- Horizontal scaling of application instances.
- Service discovery and load balancing.
- Restarting failed containers.
- Rolling application updates.
- Managing application configuration and secrets.

4.9 Situations Where a VM Is More Appropriate

A VM is more suitable when:

1. The application requires a different operating-system kernel.
2. The application depends on legacy operating-system configurations.
3. Stronger isolation between workloads is required.
4. The application needs specific kernel modules or operating-system-level customization.
5. The workload cannot be easily converted into a container.

Conclusion: The university should adopt a hybrid approach. VMs should host legacy applications and critical infrastructure services, while containers should host scalable web applications, APIs and microservices.

TASK 5 – Design Virtualized Storage and Network Infrastructure

5.1 Storage Virtualization

Storage virtualization is the process of combining storage resources from multiple physical devices into a logical storage pool. Administrators can allocate storage to virtual machines without requiring each VM to use a separate physical disk.

It improves storage utilization, management, scalability and availability.

5.2 Types of Storage

Storage Type	Description	Advantages	Limitations
DAS (Direct-Attached Storage)	Storage directly connected to an individual server.	Simple configuration and low cost.	Difficult to share between multiple servers.
NAS (Network-Attached Storage)	File-level storage accessed through a network.	Easy file sharing and centralized storage.	Performance depends on network capacity and configuration.
SAN (Storage Area Network)	Dedicated network providing block-level storage to servers.	High performance and centralized storage management.	Higher cost and configuration complexity.

Recommended approach: The university should use shared SAN or an equivalent highly available block-storage solution for critical VM disks. NAS can be used for shared documents, departmental files and backup repositories. DAS may be retained for temporary or local workloads.

5.3 Software-Defined Storage (SDS)

Software-Defined Storage separates storage management from the underlying hardware. Software controls how storage resources are pooled, allocated, replicated and protected.

Benefits include:

- Centralized storage management.
- Flexible capacity allocation.
- Improved resource utilization.
- Policy-based storage management.
- Easier expansion of storage capacity.

5.4 Important Storage Technologies

Technology	Explanation	University Application
Thin Provisioning	Allocates logical storage capacity without immediately reserving the full physical capacity.	Allows VM disks to grow as required.
Snapshots	Captures the state of a storage volume at a particular point in time.	Supports quick rollback before application updates.
Replication	Copies data from one storage system to another.	Supports recovery if the primary storage system fails.
Backup	Creates separate recoverable copies of data.	Protects student records, examination data and application files.

Snapshots should not be treated as a replacement for independent backups. A snapshot stored on the same failed storage system may also become unavailable.

5.5 Network Virtualization

Network virtualization creates logical networks independently of the physical network infrastructure. It allows administrators to configure and manage virtual network segments for different applications and users.

Important Network Technologies

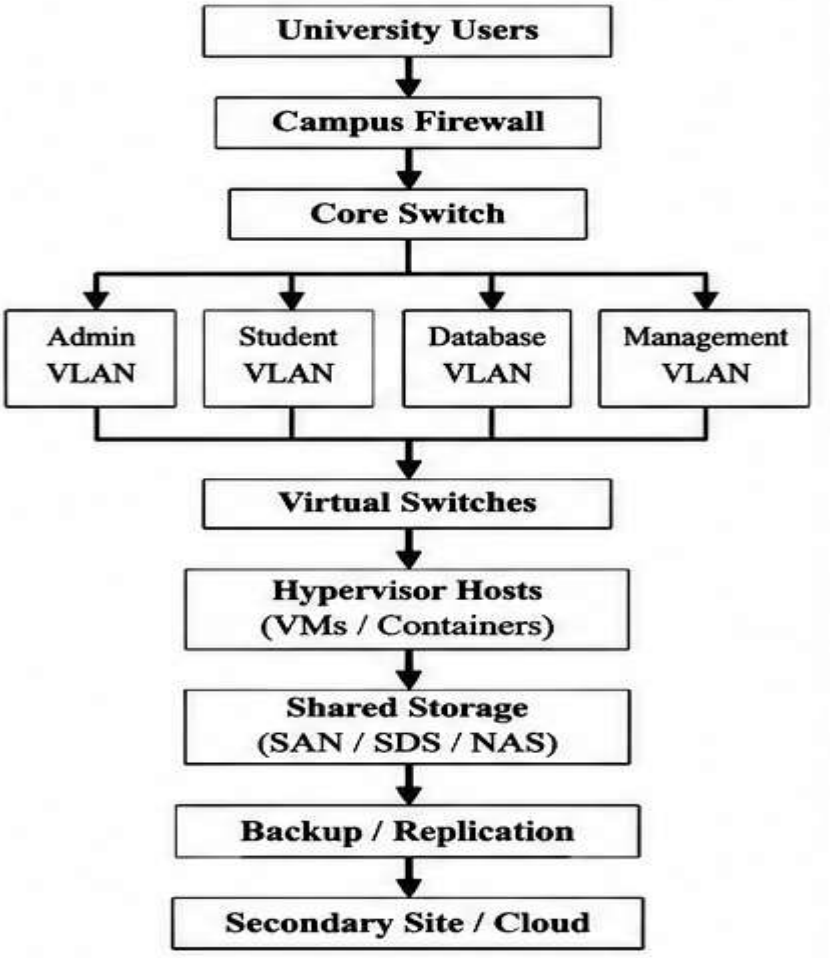
- Virtual Switch: Connects virtual machines and containers to each other and to physical networks.
- VLAN: Separates network traffic into logical broadcast domains.
- Virtual Network: Provides a logical network environment for workloads.
- Software-Defined Networking (SDN): Separates network control from packet forwarding and enables centralized network management.

5.6 Proposed Network Segmentation

The university should create separate logical network segments to improve security and control traffic.

Network Segment	Purpose	Access Policy
Administration VLAN	Administrative systems and staff access.	Accessible only to authorized staff.
Student Services VLAN	Student portal, learning platform and website traffic.	Accessible to permitted student and faculty users.
Database VLAN	Database servers and storage-related application traffic.	Accessible only from approved application servers.
Management VLAN	Hypervisor, storage, monitoring and infrastructure administration.	Restricted to infrastructure administrators.

5.7 Recommended Storage and Network Design



Conclusion: Shared storage, storage replication, logical network segmentation and virtual switches provide a flexible foundation for the university's virtualized data centre. Database traffic and management traffic should be separated from general student access.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring

6.1 Resource Allocation

Resource allocation is the process of assigning available CPU, memory, storage and network capacity to virtual machines and containers according to workload requirements.

The university should allocate resources based on application criticality, expected demand and observed utilization.

6.2 Resource Allocation for Different Resources

Resource	Allocation Method	Example
CPU	Assign virtual CPUs and configure CPU shares, reservations and limits.	Allocate more CPU capacity to the examination application during peak periods.
Memory	Assign RAM and configure memory reservations and limits.	Reserve sufficient memory for database VMs.
Storage	Allocate virtual disks from shared storage pools.	Provide higher-performance storage for database workloads.
Network	Assign virtual network interfaces and configure bandwidth policies.	Prioritize examination and student portal traffic.

6.3 Static and Dynamic Resource Allocation

Static allocation assigns fixed resources to a VM or application. The allocation remains unchanged unless an administrator modifies it.

Dynamic allocation adjusts resources according to workload requirements and configured policies.

Feature	Static Allocation	Dynamic Allocation
Resource Assignment	Fixed	Can change according to demand
Flexibility	Lower	Higher
Management	Simple to predict	Requires monitoring and policies
Utilization	May leave resources unused	Can improve resource utilization
Example	Fixed 8 GB RAM for a VM	Increasing application replicas during peak demand

6.4 Reservations, Limits and Shares

- Reservation: Guarantees a minimum amount of a resource for a VM or workload.
- Limit: Sets the maximum amount of a resource that a workload can consume.
- Shares: Define relative priority when multiple workloads compete for limited resources.

For example, the examination database can receive a higher CPU reservation than a development server.

6.5 Load Balancing

Load balancing distributes incoming requests among multiple application instances or servers.

For example, if the university examination portal runs three application instances, a load balancer can distribute user requests among those instances to avoid overloading one server.

6.6 Workload Scheduling

Workload scheduling determines where VMs or containers should run based on available CPU, memory, storage and policy requirements.

Kubernetes schedules containers to suitable worker nodes, while a hypervisor management platform schedules or manages virtual machines across physical hosts.

6.7 Auto-Scaling

Auto-scaling automatically increases or decreases the number of application instances based on measured demand.

Example:

- Normal usage: 2 application instances.
- High student traffic: Increase to 5 instances.
- Traffic decreases: Reduce the number of instances to 2.

Scaling policies should include minimum and maximum replica counts to prevent uncontrolled resource consumption.

6.8 High Availability

High availability uses redundant infrastructure and automatic recovery mechanisms to reduce service interruption.

The university can implement:

- Hypervisor host clustering.
- VM restart on another available host.
- Redundant network switches.
- Redundant power supplies and UPS systems.
- Shared storage with suitable redundancy.
- Multiple application instances behind a load balancer.

High availability does not eliminate all downtime. It reduces the impact of individual component failures.

6.9 Performance Indicators

Metric	Purpose
CPU Utilization	Measures processor usage.
Memory Utilization	Measures RAM consumption.
Disk I/O	Measures storage read and write activity.
Network Throughput	Measures the amount of data transferred.
Latency	Measures the time taken for a request or response.

VM Availability	Measures whether a virtual machine is operational and accessible.

6.10 Monitoring and Alerting Plan

The university can use Prometheus and Grafana for infrastructure monitoring. Hypervisor metrics, node metrics and application metrics should be collected centrally.

The following thresholds are proposed initial operational targets. They should be adjusted after baseline performance measurements.

Metric	Warning Threshold	Critical Threshold	Corrective Action
CPU Utilization	Above 70% for 10 minutes	Above 85% for 5 minutes	Inspect processes, rebalance workloads or scale out.
Memory Utilization	Above 75%	Above 90%	Identify memory-heavy workloads and increase capacity if required.
Disk Utilization	Above 75%	Above 90%	Expand storage or remove unnecessary data after validation.
Network Utilization	Above 70% of link capacity	Above 90%	Investigate traffic and increase bandwidth if needed.
Application Latency	Above defined service baseline	Sustained breach of application SLO	Inspect database, application and network bottlenecks.
VM Availability	Unexpected restart or failure	Critical service unavailable	Trigger failover or recovery procedure.

6.11 Response to Resource Contention

When resource contention occurs, the administrator should:

1. Identify the affected host or application.
2. Check CPU, memory, storage and network metrics.
3. Identify workloads consuming excessive resources.
4. Apply resource limits or adjust reservations where appropriate.
5. Migrate workloads or scale application instances if capacity is available.
6. Add physical or cloud capacity if the problem is sustained.
7. Record the incident and review the allocation policy.

Conclusion: A combination of dynamic resource allocation, workload scheduling, load balancing and centralized monitoring will help the university maintain performance during normal and peak periods.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery

7.1 Security Risks in a Virtualized Data Centre

Virtualization improves flexibility but introduces security risks that must be managed.

Risk	Explanation	Mitigation
VM Escape	An attacker attempts to break out of a VM and access the host or other environments.	Patch hypervisors, restrict access and use supported virtualization configurations.

VM-to-VM Attack	An attacker compromises one VM and attempts to attack another VM on the same host.	Apply network segmentation, firewall rules and VM isolation.
Insecure Snapshots	Snapshots may contain sensitive data or credentials.	Encrypt snapshots and restrict access and retention.
VM Sprawl	Uncontrolled creation of unnecessary VMs increases security and management risks.	Maintain an inventory and approval process.
Denial-of-Service	Excessive traffic or resource consumption makes services unavailable.	Use rate limiting, traffic filtering, resource limits and monitoring.
Privilege Abuse	Unauthorized users obtain administrative access.	Enforce least privilege, MFA and role-based access control.

7.2 Hypervisor Security

The hypervisor controls access to the physical server resources. If it is compromised, multiple virtual machines may be affected.

Security measures include:

- Apply regular security patches.
- Restrict management access to a dedicated network.
- Disable unnecessary services.
- Use secure administrator authentication.
- Monitor hypervisor logs.
- Restrict access to VM consoles and snapshots.

7.3 VM Isolation

VM isolation ensures that one virtual machine cannot directly access the memory, storage or resources of another VM without authorization.

The university should configure separate virtual networks and firewall policies for application, database and management workloads.

7.4 Secure VM Images

The university should maintain approved VM templates containing supported operating systems and security configurations.

Recommended controls include:

- Use trusted operating-system images.
- Remove unnecessary software and services.
- Apply security patches before deployment.
- Avoid storing passwords or private keys in templates.
- Scan images for vulnerabilities.
- Maintain version-controlled templates.

7.5 Patch Management

Patch management is the process of identifying, testing and installing security and software updates.

A controlled patching process should include:

1. Identify available updates.
2. Assess their security importance.
3. Test patches in a non-production environment.
4. Schedule maintenance windows.
5. Apply updates to production systems.
6. Validate application functionality.
7. Record patch status.

7.6 Privileged Access Control

Role-Based Access Control (RBAC) assigns permissions based on user roles.

For example:

Role	Permissions
Infrastructure Administrator	Manage hypervisors, hosts and virtual networks.
Database Administrator	Manage database services and database backups.
Application Administrator	Deploy and maintain university applications.
Auditor	View approved logs and compliance reports.
Student	Access permitted student-facing services only.

Multi-Factor Authentication (MFA) should be required for privileged administrative accounts.

7.7 Additional Security Controls

- Network Segmentation: Separates administration, database, student and management traffic.
- Encryption: Protects sensitive information in transit and at rest.
- MFA: Requires more than one authentication factor.
- RBAC: Limits access according to assigned responsibilities.
- Audit Logging: Records administrative actions, authentication events and security incidents.

7.8 Backup, Replication, Snapshots and Live Migration

Technology	Explanation	Purpose
Backup	Creates a separate recoverable copy of data.	Protects against deletion, corruption and system failure.
Replication	Copies data to another storage system or site.	Supports recovery after primary storage failure.
Snapshot	Captures the state of a VM or storage volume at a specific time.	Supports rollback and short-term recovery.
Live Migration	Moves a running VM from one physical host to another with minimal interruption when supported and properly configured.	Enables maintenance and workload balancing.
Disaster Recovery	Restores applications and data after a major disruption.	Maintains business continuity.

7.9 RTO and RPO

Recovery Time Objective (RTO): The maximum acceptable time taken to restore a service after an interruption.

Recovery Point Objective (RPO): The maximum acceptable amount of data loss measured in time.

For example, an RPO of 15 minutes means the organization aims to restore data to a point no more than 15 minutes before the incident.

7.10 Proposed RTO and RPO Targets

The following are proposed targets for planning. The university should confirm them with application owners based on business impact and actual recovery costs.

Application	Proposed RTO	Proposed RPO	Justification
Student Information System	1 hour	15 minutes	Student records and academic information are important institutional data.
Examination Portal	30 minutes	5 minutes	Examination availability and submitted responses are time-sensitive.
Library Management System	4 hours	1 hour	Temporary interruption may be manageable, but records must be recoverable.
Institutional Website	4 hours	1 hour	The website can tolerate a longer recovery period than examination services.

7.11 Disaster Recovery Plan

The university should implement the following recovery process:

1. Maintain regular backups of application data and VM configurations.
2. Replicate critical data to a separate recovery location.
3. Monitor backup completion and replication health.
4. Document recovery procedures for each application.
5. Define roles and responsibilities during an incident.
6. Test restoration procedures periodically.
7. Record recovery times and compare them with the approved RTO and RPO.
8. Update the recovery plan after major infrastructure changes.

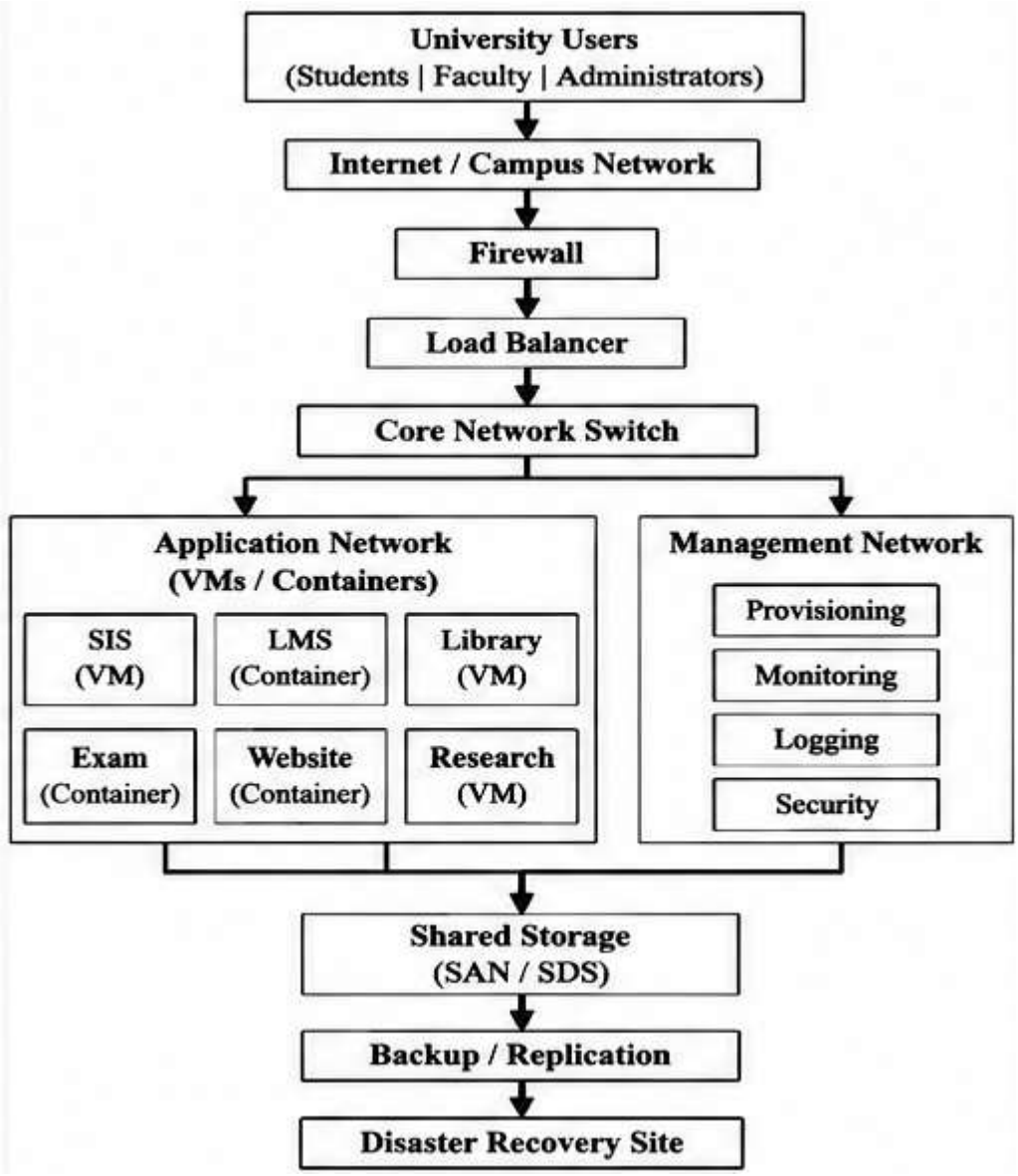
Conclusion: Security controls, redundant infrastructure, tested backups and documented recovery objectives are necessary to protect the university's applications and data.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture

8.1 Proposed Architecture

The proposed architecture uses a combination of virtual machines and containers, shared storage, virtual networking, centralized management and a secondary recovery location.

8.2 Labelled Architecture Diagram



8.3 Architecture Components and Their Functions

Component	Function
Users	Students, faculty and administrators access university services.
Internet/Campus Network	Provides connectivity to internal and external users.
Firewall	Filters unauthorized network traffic.
Load Balancer	Distributes requests across application instances.
Virtual Switches	Connect VMs and containers to logical networks.
Hypervisor Hosts	Provide physical resources for virtual machines.
VMs	Run legacy systems, databases and operating-system-dependent applications.
Containers	Run scalable web applications, APIs and microservices.
Shared Storage	Provides centralized storage for virtual machines and application data.
Management Plane	Supports provisioning, monitoring, logging and policy enforcement.
Backup/Replication	Protects application data and supports recovery.
Secondary Site	Provides an alternate recovery location during a major outage.

8.4 Distribution of University Applications

Application	Deployment	Reason
Student Information System	VM	Supports legacy application requirements.
Database Services	VM	Provides controlled resource allocation and isolation.
Library Management System	VM or Container	Depends on application compatibility and architecture.
Institutional Website	Containers	Supports independent scaling of web components.
Learning Management System	Containers and VMs	Web components can scale independently while supporting required backend services.
Examination Portal	Containers with database VMs	Supports increased application traffic and controlled database access.
Research Applications	VM or Container	Selected according to operating-system and software dependencies.

8.5 Management Plane

The management plane provides centralized control of the infrastructure.

It should support:

- VM and container provisioning.
- Resource allocation and policy enforcement.
- Performance monitoring.
- Centralized logging.
- Security configuration.
- Backup management.
- Capacity planning.

8.6 Scalability

The proposed architecture supports scalability through:

1. Increasing VM resources when host capacity is available.
2. Adding physical hosts to the virtualization cluster.
3. Increasing container replicas during high traffic.
4. Using load balancing to distribute requests.
5. Using cloud capacity for temporary demand where financially and technically suitable.

8.7 Performance

Performance is improved through:

- CPU and memory allocation based on workload requirements.
- Shared high-performance storage.
- Network segmentation.
- Application-level load balancing.
- Monitoring of CPU, memory, disk I/O and network latency.

8.8 Fault Tolerance

Fault tolerance is supported by:

- Multiple hypervisor hosts.
- Host failover.
- Redundant network paths.
- Redundant storage components.
- Backup and replication.
- Recovery infrastructure at a secondary site.

8.9 Security

Security is provided through:

- Firewalls.
- Network segmentation.
- Encryption.
- MFA.
- RBAC.
- Secure VM templates.
- Centralized audit logging.

8.10 Cost Efficiency

The architecture reduces unnecessary physical server usage and allows infrastructure resources to be allocated according to demand. However, shared storage, redundant hosts, licensing and disaster-recovery infrastructure create additional costs. Therefore, the final design should be evaluated using a three-year cost model.

Conclusion: The proposed architecture integrates compute, storage, networking, virtualization and management into a centralized infrastructure. It provides a foundation for improved resource utilization, application scalability and recovery.

TASK 9 – Hands-on Implementation and Validation

9.1 Objective

The objective of this practical implementation is to develop, deploy and validate a university-related Student Records REST API using Docker. The application was successfully packaged and executed inside a Docker container. The implementation demonstrates container deployment, resource configuration, functional API testing and resource monitoring.

The application provides endpoints to check service health and retrieve student information. It is implemented using Python's built-in HTTP server and standard-library modules; it does not use Flask. Docker is used to package and run the application in an isolated container.

9.2 Tools and Technologies

Tool	Purpose
Ubuntu / WSL2	Development and execution environment
Docker	Containerization platform
Docker Compose	Container configuration and management
Python	Application programming language

Python standard-library HTTP server	REST API implementation
curl	Functional API testing
Docker Stats	Actual Docker container CPU, memory, network and process monitoring.

9.3 Implementation Files

The project contains:

```
university-cloud-poc/
├── app/
│   └── app.py
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── tests/
│   └── test_api.sh
└── README.md
```

9.4 Application Description

The Python standard-library HTTP server application provides the following API endpoints:

Endpoint	Method	Purpose
/health	GET	Checks whether the application is running.
/api/students	GET	Retrieves the student records.
/api/students/<id>	GET	Retrieves a particular student record.

The application uses sample student data for demonstration. It is not a production student-information system and does not include authentication or persistent database storage.

9.5 Implementation Procedure

Step 1: Download and extract the project

Extract the project and open PowerShell in the directory containing compose.yaml. Docker Desktop must be running before executing the following commands.

Navigate to the project directory:

```
cd university-cloud-poc
docker compose up -d --build
docker compose ps
curl.exe http://localhost:5001/health
curl.exe http://localhost:5001/api/students
curl.exe http://localhost:5001/api/students/1
curl.exe -i http://localhost:5001/api/students/999
```

9.6 Resource Configuration

The supplied Docker Compose file defines the following resource limits. These are configured values, not measurements from a running container.

Resource	Configured Value
CPU Limit	1 CPU
Memory Limit	512 MB
Container Port	5001
Application Port	5000

The host port was changed to 5001 because port 5000 was already occupied by another running container. Docker maps host port 5001 to application port 5000 inside the university-student-api container.

9.7 Docker Container Resource Monitoring

The resource utilization of the Student Records API was monitored using Docker's built-in resource monitoring command. The Docker container was successfully started using Docker Compose with the container name university-student-api.

The following command was used to monitor the resource utilization of the running container:

docker stats --no-stream university-student-api

Observed Docker Resource Utilization:

The following output was obtained from the running Docker container.

CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM % NET I/O BLOCK I/O PIDS
45ce9c71fda3 university-student-api 0.01% 13.59MiB / 512MiB 2.66% 2.97kB / 1.67kB 0B / 0B 1

Resource Monitoring Results:

Resource	Observed Value
Container Name	university-student-api
CPU Utilization	0.01%
Memory Usage	13.59 MiB
Memory Limit	512 MiB
Memory Utilization	2.66%
Network Input	2.97 kB
Network Output	1.67 kB
Block I/O	0 B / 0 B
Running Processes (PIDs)	1

Analysis:

1. The Docker container was running successfully during resource monitoring.
2. CPU utilization was recorded as 0.01%, indicating very low CPU consumption at the time of measurement.
3. Memory consumption was 13.59 MiB out of the configured 512 MiB limit.
4. Memory utilization was 2.66%, showing that the application was operating within its configured memory limit.
5. Network input and output values were recorded by Docker during the monitoring snapshot.
6. The container reported one running process.

The results demonstrate that Docker successfully monitored the resource consumption of the Student Records API container. The application remained within its configured memory allocation during the captured measurement.

9.8 Testing and Validation Results

The Student Records API was deployed using Docker Compose. The container execution status was checked using Docker Compose, and resource utilization was verified using the Docker statistics command.

1. Docker Container Execution Verification

The following command was executed:

```
docker compose ps
```

The following output confirmed that the application container was running:

```
NAME IMAGE COMMAND SERVICE STATUS
university-student-api university-cloud-poc-student-records-api "python app.py" student-records-api Up
```

The port mapping was:

```
0.0.0.0:5001->5000/tcp
```

This confirms that the application running on port 5000 inside the container was accessible through port 5001 on the host machine.

2. Docker Resource Monitoring Verification

The following command was used:

```
docker stats --no-stream university-student-api
```

The container reported:

- CPU utilization: 0.01%
- Memory usage: 13.59 MiB
- Configured memory limit: 512 MiB
- Memory utilization: 2.66%

This confirms that Docker resource monitoring was successfully performed on the running container.

3. API Functional Testing

The following endpoints were used to validate the Student Records API through the Docker container.

Test Case	API Endpoint	Expected Result	Actual Result
Health Check	/health	HTTP 200 – Service available	Pending Docker verification

Retrieve All Students	/api/students	HTTP 200 – Student records in JSON	Pending Docker verification
Retrieve Individual Student	/api/students/1	HTTP 200 – Individual student details	Pending Docker verification
Invalid Student ID	/api/students/999	HTTP 404 – Student not found	Pending Docker verification

Commands Used for API Testing:

curl.exe http://localhost:5001/health
curl.exe http://localhost:5001/api/students
curl.exe http://localhost:5001/api/students/1
curl.exe -i http://localhost:5001/api/students/999

4. Validation Summary

Validation Parameter	Result
Docker Image	Successfully built
Docker Container	Running
Docker Compose	Successfully executed
Container Port Mapping	Verified – 5001:5000
CPU Resource Monitoring	Verified
Memory Resource Monitoring	Verified
Memory Limit	512 MiB configured
API Functional Testing	Requires Docker endpoint test results

Conclusion:

The Student Records API was successfully containerized and executed using Docker Compose. The running container was verified using docker compose ps, and its CPU, memory, network, and process utilization were monitored using docker stats. The container operated within the configured 512 MiB memory limit during the captured snapshot. The API endpoint tests must also be completed against the Docker host port to establish full functional validation.

9.9 Actual Screenshot Evidence

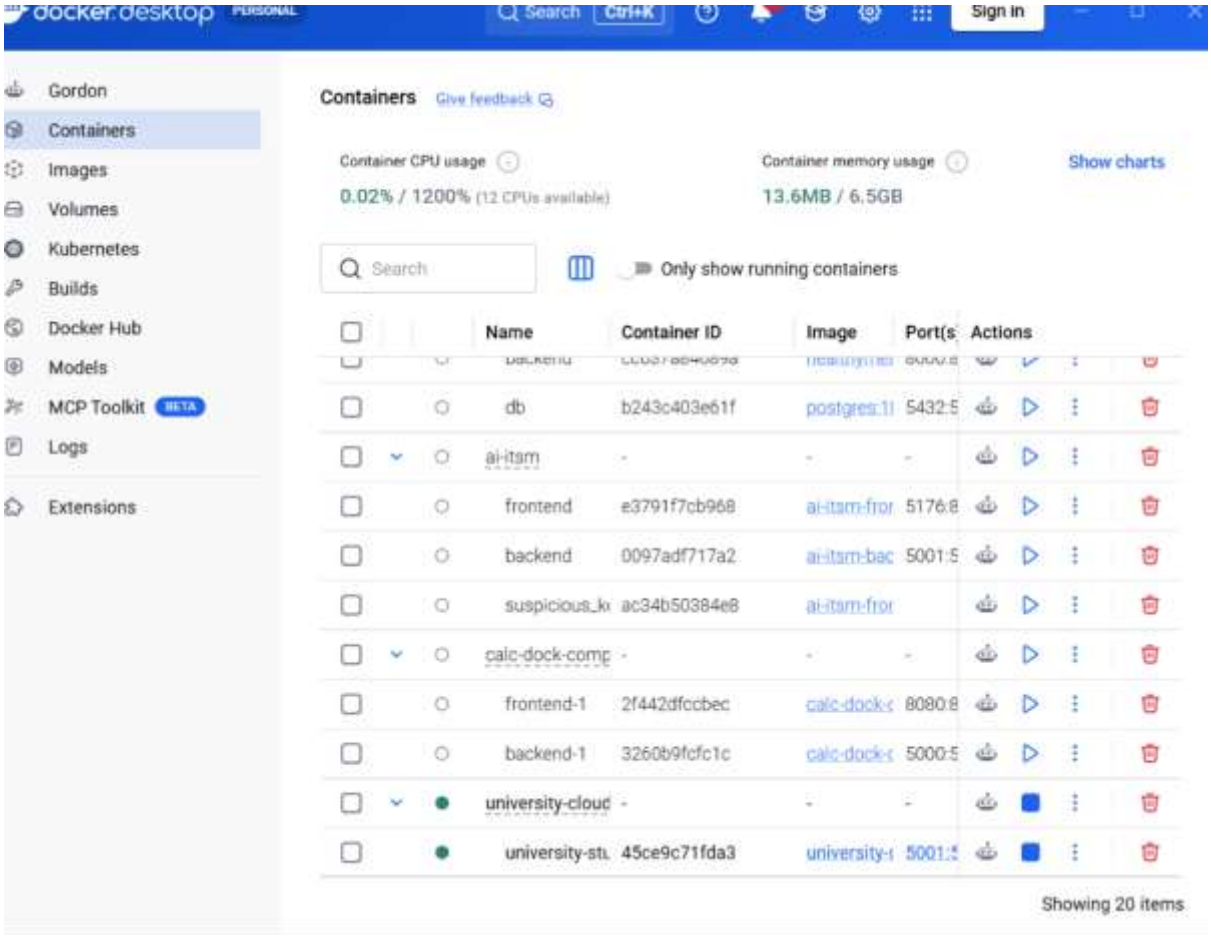


Figure 9.1 - Docker Container Running

File Edit View

```
services:
  student-records-api:
    build: .
    container_name: university-student-api
    ports:
      - "5001:5000"
    environment:
      HOST: 0.0.0.0
      PORT: 5000
    cpus: 1.0
    mem_limit: 512m
    restart: unless-stopped
```

Figure 9.2 – Docker Compose resource configuration

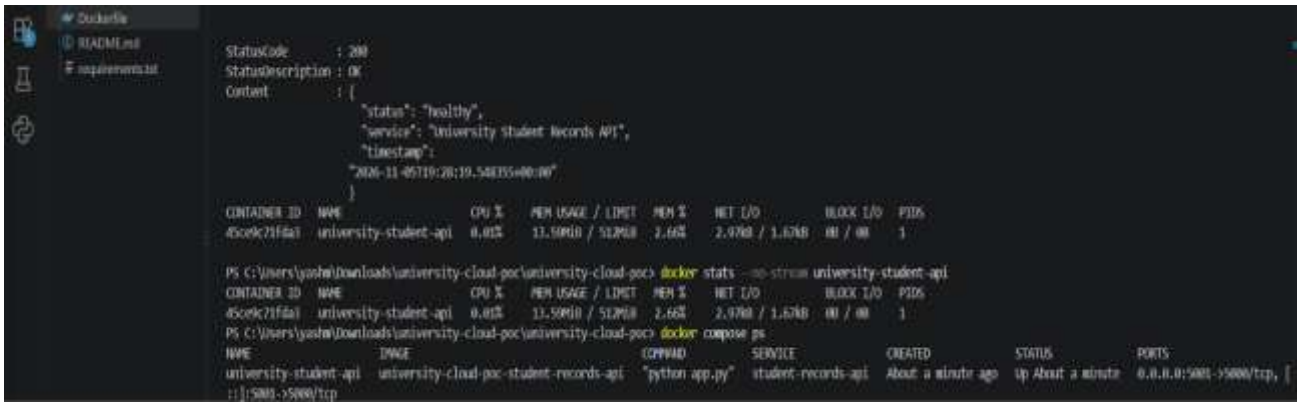


Figure 9.3 - Docker Resource Usage



Figure 9.4 - Docker Health Check



Figure 9.5 - Docker Student API Response



Figure 9.6 - Docker Individual Student Details

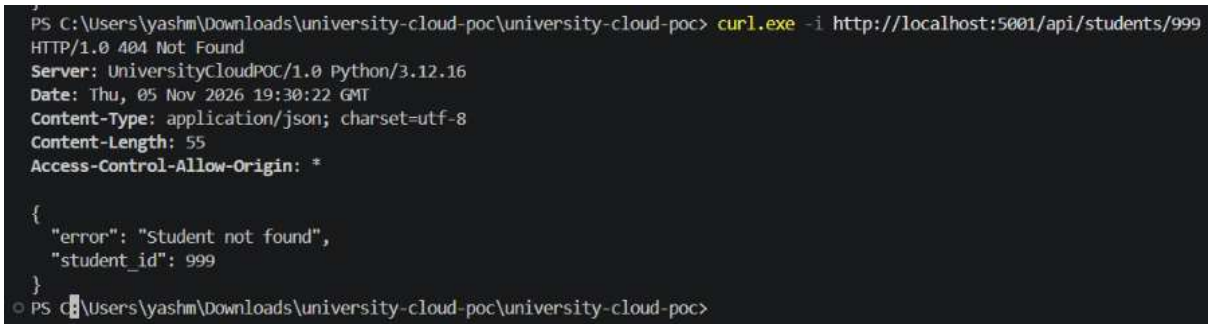


Figure 9.7 - Docker Invalid Student 404

9.10 Implementation Outcome

The proof of concept was successfully deployed inside a Docker container named university-student-api. Docker Compose was used to configure and start the container with a CPU limit of 1 CPU and a memory limit of 512 MB. The container was verified as running, and actual CPU, memory and network resource measurements were obtained using Docker Stats. Functional API test results are documented separately based on the responses obtained from the running container.

TASK 10 – Evaluate Cost, Performance and Final Recommendation

10.1 Comparison of Infrastructure Models

The existing physical-server arrangement is compared with a virtualized on-premises model and a hybrid-cloud model.

Parameter	Physical Infrastructure	Virtualized On-Premises	Hybrid Cloud
Hardware	Separate physical servers	Consolidated virtualization hosts	On-premises core plus cloud resources
Electricity	Higher	Potentially lower per workload	Lower on-premises requirement, but cloud operating costs apply
Cooling	Higher due to more physical servers	Potentially lower	Depends on remaining on-premises equipment
Maintenance	More physical equipment to maintain	Centralized infrastructure management	Shared on-premises and cloud responsibilities
Licensing	Operating-system and application licensing	Hypervisor, OS and application licensing	Cloud service and application licensing
Scalability	Requires hardware procurement	Add hosts or allocate resources	Can provision cloud resources when required
Availability	Depends on individual physical servers	Can use clustering and failover	Can use redundant on-premises and cloud services
Administration	Separate server administration	Centralized management	Requires management of both environments

10.2 Three-Year Cost Comparison

The assignment does not provide actual equipment prices, electricity tariffs, maintenance contracts or cloud-provider quotations. Therefore, the following is an illustrative planning model, not a verified market quotation.

All values are expressed in ₹ lakh.

Assumptions

- The physical infrastructure currently contains 20 servers.
- The virtualized design uses 16 hosts for the existing peak-capacity requirement calculated in Task 3.
- The hybrid model retains a smaller on-premises core and uses cloud capacity for selected variable workloads.
- All three options are compared over three years.
- Hardware, support and cloud costs are assumed for academic comparison only.
- Actual procurement quotations and electricity bills must replace these assumptions before financial approval.

Estimated Cost Table

Cost Category	Physical Model	Virtualized Model	Hybrid Model
Existing/new server hardware	100	128	64
Shared storage and network	0	40	20
Migration and initial setup	0	15	10
One-time capital/setup cost	100	183	94
Annual electricity and cooling	24	18	12
Annual maintenance/support	12	15	8
Annual licensing/cloud services	6	6	45
Annual administration	6	6	5
Annual recurring cost	48	45	70
Three-year recurring cost	144	135	210
Total three-year cost	244	318	304

Calculation

Physical infrastructure:

$$100 + (48 \times 3) = 244 \text{ lakh}$$

Virtualized infrastructure:

$$183 + (45 \times 3) = 318 \text{ lakh}$$

Hybrid infrastructure:

$$94 + (70 \times 3) = 304 \text{ lakh}$$

All figures are illustrative assumptions, not actual quotations. The hybrid model's cloud cost depends heavily on workload volume, storage, data transfer and provider pricing.

10.3 Cost Analysis

The virtualized infrastructure has a higher estimated three-year cost in this example because the initial investment includes 16 physical hosts, shared storage, network infrastructure and migration.

However, it provides centralized management and the ability to consolidate workloads.

The hybrid model has lower initial setup costs but higher recurring cloud-service expenses. It may be financially attractive when demand is variable and the university can avoid purchasing hardware for occasional peak workloads.

The physical model has lower assumed initial costs in this comparison because it represents the existing infrastructure, but it has less flexibility and may require additional hardware investment as demand increases.

Therefore, virtualization should not be justified only by assuming that it always reduces total cost. Its financial benefits depend on hardware utilization, licensing, support, energy consumption and workload growth.

10.4 Expected Performance Improvements

The proposed virtualized infrastructure is expected to provide:

1. Improved utilization: Shared resources can be allocated to workloads according to their requirements.
2. Scalability: Additional VMs or containers can be deployed when capacity is available.
3. Availability: Clustering and failover can reduce the impact of individual host failures.
4. Manageability: Centralized tools simplify provisioning and monitoring.
5. Flexibility: Applications can be migrated or resized more easily than applications tied to dedicated physical servers.

10.5 Risks and Mitigation

Risk	Possible Impact	Mitigation
Licensing Costs	Unexpected operating expenses.	Evaluate open-source and commercial alternatives before procurement.
Vendor Lock-In	Difficulty moving workloads to another provider.	Prefer portable VM images, container images and standard interfaces.
Skills Requirements	Administrators may lack virtualization and cloud expertise.	Provide technical training and operational documentation.
Hardware Failure	Multiple workloads may be affected by one host failure.	Use clustering, redundant hardware and tested failover.
Migration Downtime	Services may become unavailable during migration.	Use phased migration and planned maintenance windows.
Storage Failure	Loss of access to multiple virtual machines.	Use redundant storage and independent backups.

10.6 Phased Migration Roadmap

Phase	Activity	Reason
Phase 1	Infrastructure assessment and capacity planning.	Identify application dependencies and resource requirements.

Phase 2	Install hypervisor hosts and shared storage.	Establish the virtualization environment.
Phase 3	Migrate development and testing workloads.	Validate the platform with low-risk applications.
Phase 4	Migrate institutional website and selected library services.	Test production workloads with manageable impact.
Phase 5	Migrate student and learning applications.	Move important services after infrastructure validation.
Phase 6	Migrate examination and critical database workloads.	Perform carefully planned migration and data validation.
Phase 7	Configure backup, replication and disaster recovery.	Improve recovery capability.
Phase8	Monitor, optimize and document.	Validate performance, security and operating costs.

10.7 Final Recommendation

The university should adopt a virtualized on-premises infrastructure with a hybrid-cloud extension for selected workloads.

KVM-based virtualization, shared storage, Docker containers, centralized monitoring and backup replication provide a flexible foundation.

The university should not immediately migrate every application. A phased approach allows administrators to validate performance, compatibility, security and recovery procedures before moving critical applications.

The final purchasing decision should be based on actual vendor quotations and measured workload requirements.

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure

11.1 Changed Requirements

The university introduces a new digital learning platform and extends selected services to 10 affiliated colleges.

The changed requirements are:

- Peak workload increases to five times the current peak.
- At least 99.9% service availability is required.
- Secure isolation between affiliated colleges is necessary.
- Storage and network capacity must support additional users.
- Infrastructure costs must remain controlled.

11.2 Impact Analysis

A. Workload Changes

The new digital learning platform will increase the number of concurrent users, application requests, database operations and file transfers.

The university must support both its original applications and the additional learning platform traffic.

B. Storage Changes

More users will generate additional:

- Learning materials.
- Assignment submissions.
- Examination records.
- Multimedia content.
- Application logs.
- Database records.

The university must therefore plan for additional storage capacity and backup requirements.

C. Network Traffic Changes

Extending services to affiliated colleges increases traffic between users, application services and databases.

The university should provide sufficient network bandwidth, traffic monitoring, load balancing and secure connectivity.

D. Availability Changes

The original infrastructure did not have a tested disaster-recovery site or documented recovery objectives.

The new 99.9% availability requirement requires stronger redundancy, monitoring, failover and recovery procedures.

E. Multi-Tenant Security

The affiliated colleges should be treated as separate tenants.

Each college must have access only to its authorized services and data.

Recommended controls include:

- Separate tenant identities.
- Role-based access control.
- Tenant-specific database authorization.
- Network segmentation.
- Separate namespaces or equivalent workload boundaries.
- Centralized audit logging.

The system must enforce tenant isolation at the application, network and data layers.

11.3 Capacity and Availability Evaluation

The current total CPU capacity is:

$$20 \times 16 = 320 \text{ cores}$$

The existing peak demand at 95% utilization is:

$$320 \times 0.95 = 304 \text{ cores}$$

The new expected peak is five times the existing peak.

$$304 \times 5 = 1520 \text{ cores}$$

Therefore, the estimated new peak CPU demand is:

$$\boxed{1520 \text{ cores}}$$

11.4 Revised CPU Host Calculation

Assume each new physical host has 32 CPU cores and the target CPU utilization remains 65%.

Usable CPU capacity per host:

$$32 \times 0.65 = 20.8 \text{ cores}$$

Required active hosts:

$$N = \frac{1520}{20.8}$$

$$N = 73.08$$

Rounding up:

$$\boxed{74 \text{ active hosts}}$$

To provide capacity for one host failure:

$$74 + 1 = \boxed{75 \text{ hosts}}$$

Therefore, under this simplified CPU-sizing assumption, the university would require 75 hosts to support five times the current 95% peak CPU demand while retaining one additional host as an N+1 reserve.

This is a planning estimate, not a final procurement quantity. Actual sizing must also account for workload concurrency, memory, storage performance, network capacity and measured application demand.

11.5 Memory Capacity Estimate

The earlier Task 3 calculation assumed average memory utilization of 60%.

Estimated average memory demand:

$$1280 \times 0.60 = 768 \text{ GB}$$

For this simplified estimate, assume the memory demand increases in the same proportion as peak CPU demand.

Five-times estimated memory demand:

$$768 \times 5 = 3840 \text{ GB}$$

Assuming 256 GB RAM per host and 25% memory reserve:

$$256 \times 0.75 = 192 \text{ GB}$$

Required active hosts:

$$N = \frac{3840}{192} = 20$$

Adding one host for N+1 capacity:

$$20 + 1 = \boxed{21 \text{ hosts}}$$

This is an indicative memory calculation based on the earlier 60% memory-utilization assumption. It does not establish the actual peak memory requirement because the assignment provides no memory-utilization measurements.

Under these assumptions, CPU capacity is the limiting factor rather than memory.

11.6 Storage and Network Capacity

The assignment does not provide storage-utilization or network-throughput measurements. Therefore, an exact storage or bandwidth requirement cannot be calculated from the supplied data.

The university should use the following planning process:

1. Measure current storage utilization and annual growth.
2. Identify the storage required by the new learning platform.
3. Estimate additional space for assignments, documents, videos and logs.
4. Calculate backup and replication overhead.
5. Measure peak network throughput during admission and examination periods.
6. Include additional capacity for traffic from all affiliated colleges.
7. Validate the design using load tests.

Storage and network capacity should be increased based on measured workload growth rather than an unsupported fixed multiplier.

11.7 Meaning of 99.9% Availability

Availability represents the percentage of time a service remains operational.

For 99.9% availability:

$$\text{Downtime Percentage} = 100 - 99.9 = 0.1\%$$

Annual downtime:

$$365 \times 24 \times 60 \times 0.001$$

$$= 525.6 \text{ minutes}$$

$$\boxed{8.76 \text{ hours per year}}$$

Therefore, the service must be designed to keep annual downtime within approximately 8.76 hours.

This target requires redundancy and operational processes; a single server or single storage device is insufficient.

11.8 Measures to Achieve 99.9% Availability

The university should implement:

- Multiple hypervisor hosts.
- Automatic VM or application failover.
- Redundant switches and network paths.
- Redundant storage.
- Load-balanced application instances.
- Regular backups and replication.
- Monitoring and alerting.
- Tested disaster-recovery procedures.
- Planned maintenance and patching procedures.

A secondary recovery site should be designed according to the approved RTO and RPO. The university must test recovery procedures rather than relying only on backup completion reports.

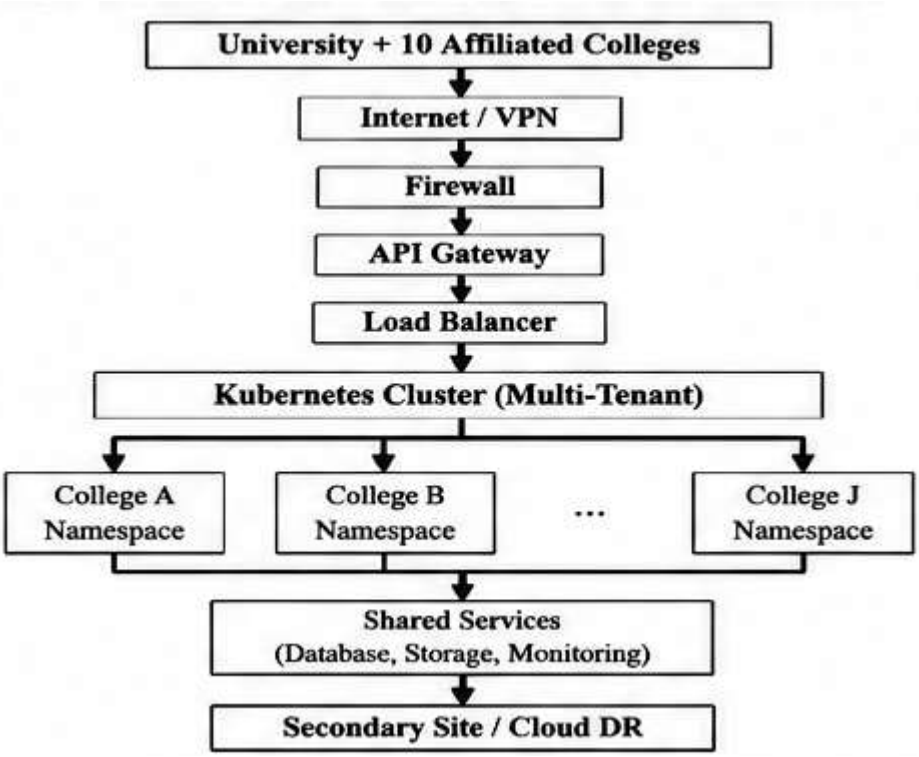
11.9 Adaptation or Redesign

Recommendation: Modify the original design

The original virtualization architecture should be modified rather than retained without changes or completely replaced.

The original design provides a useful foundation, but the five-times peak workload and multi-tenant requirement require additional capacity, stronger isolation and improved availability.

Revised Architecture



11.10 Revised Virtualization and Resource Allocation

The revised architecture should include:

1. Virtualization clustering: Multiple hosts should operate as a managed cluster to support workload distribution and failover.
2. Container orchestration: Kubernetes should manage the deployment and scaling of the learning platform's application services.
3. Auto-scaling: Application replicas should increase when traffic exceeds defined thresholds.
4. Resource reservations: Critical database and examination workloads should receive appropriate minimum resource guarantees.
5. Resource limits: Each tenant's workloads should have CPU and memory limits to prevent one college from exhausting shared resources.
6. Live migration: Supported VMs should be migrated between hosts for maintenance and load balancing.
7. Hybrid-cloud capacity: Temporary or variable workloads may use cloud resources if the cost and data-security requirements permit.

11.11 Multi-Tenant Isolation

Each affiliated college should be provided with a logically isolated environment.

The university can implement:

- Separate Kubernetes namespaces.
- Tenant-specific service accounts.
- Role-based access control.
- Network policies between namespaces.
- Tenant-specific application authorization.
- Separate database schemas or databases where appropriate.
- Encryption of sensitive data.

- Centralized audit logs.

Namespaces alone do not provide complete security isolation. Network policies, identity controls, application authorization and database access restrictions must also be implemented.

11.12 Service Model Changes

The revised design can use a combination of cloud service models.

Service Model	University Application
IaaS	Virtual machines, networking and storage infrastructure.
PaaS	Managed application platforms and databases where suitable.
SaaS	Third-party services such as email, collaboration or learning tools where appropriate.

The university should retain control of critical application data and identity management while using managed services when they provide measurable operational or financial benefits.

11.13 Cost and Operational Trade-Offs

The revised architecture increases hardware and operational complexity because it must support a much larger workload.

Area	Expected Impact	Management Approach
Performance	Higher resource demand.	Capacity planning and load testing.
Availability	More redundancy is required.	Clustering and secondary-site recovery.
Security	More tenants create additional isolation requirements.	Tenant-specific identity and network policies.
Licensing	More hosts and managed services may increase licensing costs.	Compare open-source and commercial alternatives.
Management	Kubernetes and multi-site operations increase complexity.	Training, automation and centralized monitoring.
Migration	Additional services require controlled migration.	Phased deployment and rollback plans.

11.14 Important Note: Alternative Ten-Times / 40-College Interpretation

The assignment contains a repeated section that refers to ten times the current peak workload and 40 colleges. This conflicts with the main changed requirement of five times the current peak and 10 colleges.

If the evaluator expects the later ten-times scenario, the CPU calculation becomes:

$$304 \times 10 = 3040 \text{ cores}$$

Required active hosts:

$$N = \frac{3040}{20.8} = 146.15$$

Rounding up:

147 active hosts

Adding one N+1 host:

$$147 + 1 = \boxed{148 \text{ hosts}}$$

The ten-times scenario would therefore require approximately 148 hosts under the same host specification and CPU-utilization assumptions.

This alternative calculation should not be confused with the main five-times requirement.

11.15 Final Decision

The university should modify the original virtualized infrastructure by introducing a clustered environment, container orchestration, API gateway, multi-tenant isolation, automated scaling and secondary-site disaster recovery.

The five-times workload calculation indicates a requirement of approximately 75 physical hosts under the specified 32-core host and 65% target-utilization assumptions, including one host for N+1 capacity.

However, final procurement should be based on actual workload testing and detailed CPU, memory, storage and network measurements.

The revised architecture provides improved scalability, availability, security and resource management while allowing the university to control costs through phased implementation and selective use of hybrid-cloud services.

Overall Conclusion

The analysis demonstrates that virtualization can improve the university's data-centre management by allowing multiple workloads to share physical infrastructure.

The proposed design combines hypervisors, virtual machines, containers, shared storage, virtual networking, centralized monitoring and disaster recovery.

The calculations indicate that the existing infrastructure has 320 CPU cores, but its average CPU demand is only 64 core-equivalents. Consolidation can improve resource utilization during normal operations. However, peak demand must be considered when deciding the final number of hosts.

The proposed hybrid approach provides a flexible foundation for supporting the university's existing applications and its new digital learning platform. With suitable capacity planning, security controls, monitoring and disaster-recovery procedures, the university can improve service reliability and prepare for future growth.